SOFTWARE ENGINEERING + DEVSECOPS PORTFOLIO PROJECT

DeployShield

Automated Infrastructure Security & Compliance Auditing Platform

Go Master Guide

Four-month development roadmap and build blueprint

Core stack Go • PostgreSQL • GitHub Actions • Terraform • Kubernetes • Docker • OPA/Rego • Argo CD • Prometheus • Grafana

Prepared for: **Warusha Shamalka Rupasinghe** | 23001641

BSc in Computer Science — University of Colombo School of Computing (UCSC)

Target: Software Engineering / DevOps & Cloud Internships

Contents

1 Orientation: How to Use This Guide **2**

2 Part 1 — Concept, Principle, and Go Choice **2**
- 2.1 What DeployShield Actually Does . . . 2
- 2.2 The Principle to Internalize: Config Is Input, Not a Deployment Target . . . 2
- 2.3 Go Adaptation of the Original Stack . . . 2

3 Part 2 — Rule Engine Architecture **3**
- 3.1 Pipeline . . . 3
- 3.2 Rule Definition Format . . . 4
- 3.3 Adding Rules Without Touching the Scanner . . . 5
- 3.4 Initial Rule Set: Illustrative, Not Exhaustive . . . 5

4 Part 3 — Development Phases: 0 to 14 **5**
- 4.1 Phase 0 — Project Setup . . . 6
- 4.2 Phase 1 — Core Static Scanner . . . 6
- 4.3 Phase 2 — CLI . . . 6
- 4.4 Phase 3 — REST API . . . 7
- 4.5 Phase 4 — PostgreSQL and Audit History . . . 7
- 4.6 Phase 5 — Asynchronous Scanning with Redis . . . 7
- 4.7 Phase 6 — Docker . . . 7
- 4.8 Phase 7 — CI/CD with GitHub Actions . . . 7
- 4.9 Phase 8 — GitHub Pull-Request Security Gate . . . 8
- 4.10 Phase 9 — OPA / Rego Policy Engine . . . 8
- 4.11 Phase 10 — Kubernetes . . . 8
- 4.12 Phase 11 — Kubernetes Admission Controller . . . 8
- 4.13 Phase 12 — GitOps with Argo CD . . . 8
- 4.14 Phase 13 — Observability . . . 8
- 4.15 Phase 14 — Terraform for DeployShield's Own Infrastructure . . . 8

5 Part 4 — Your First Session Tonight (2–4 Hours) **9**

6 Part 5 — Testing Strategy **9**
- 6.1 Core Idea . . . 9

7 Part 6 — Recommended Build Order and Four-Month Plan **10**
- 7.1 Dependency-Driven Order . . . 10
- 7.2 Four-Month Calendar . . . 11

8 Part 7 — Technology Learning Map **11**

9 Part 8 — Project Management Practices **12**

10 Part 9 — Security of DeployShield Itself **12**

11 Part 10 — Real-World Positioning and Compliance **12**
- 11.1 This Tool Category Is Real . . . 13
- 11.2 Who Needs This . . . 13
- 11.3 Compliance Disclaimer: Use Precise Language . . . 13

12 Part 11 — MVP vs Advanced Scope **14**

13 Part 12 — Common Mistakes to Avoid **14**

14 Part 13 — CV Value **14**
- 14.1 Skills Demonstrated . . . 14
- 14.2 CV-Ready Description . . . 15

15 Part 14 — Success Criteria and Master Checklist **15**
- 15.1 MVP Success . . . 15
- 15.2 Advanced Success . . . 15
- 15.3 Final Master Checklist . . . 16

Orientation: How to Use This Guide

This is a build guide, not a lecture. Start with **Part 4: Your First Session** if you want to write code immediately; return to the architecture sections when you need context.

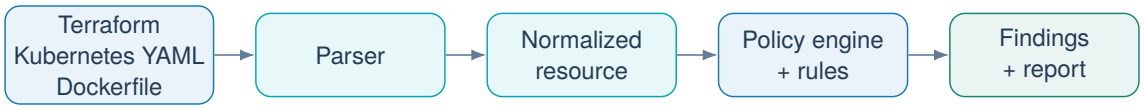
Part	What it gives you
1	Core concept, the "config is input" principle, and the Go implementation decision
2	Rule-engine architecture — the heart of the project
3	Phase-by-phase build plan, Phases 0 through 14
4	An exact 2–4 hour first coding session
5	Testing strategy: fixtures, Go test layout, and what to validate
6	Recommended build order and a practical four-month schedule
7	Technology learning map: learn now versus defer
8	Git, issues, ADRs, releases, and documentation habits
9	Security of DeployShield itself when scanning untrusted repositories
10	Real-world positioning and the compliance disclaimer
11–14	MVP boundary, mistakes to avoid, CV value, success criteria, and master checklist

CORE RULE FOR THE WHOLE PROJECT
Build the scanner before you build infrastructure to scan. Terraform files, Kubernetes YAML, and Dockerfiles are **input text**, not targets that must be deployed. Docker and Kubernetes only enter later to package and run DeployShield itself; they are not prerequisites for static analysis.

Part 1 — Concept, Principle, and Go Choice

What DeployShield Actually Does

DeployShield reads infrastructure-as-code (IaC) files and reports security problems before the described infrastructure is created. It is a static-analysis tool for infrastructure, in the same family as a source-code linter.



Every finding should carry a rule ID, description, severity (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, or `INFO`), file path, line number where determinable, remediation suggestion, and scan timestamp.

The Principle to Internalize: Config Is Input, Not a Deployment Target

A `.tf` file describes desired infrastructure. A Kubernetes manifest is YAML. A Dockerfile is a build-instruction list. DeployShield reads and interprets these files; it does not execute them. Therefore Phases 0–5 can run on an ordinary Linux machine with Go and PostgreSQL installed. No cloud account, running cluster, Docker daemon, or `terraform apply` is required for the scanner.

WHY THIS SEQUENCE MATTERS
You build working, testable software before touching Docker or Kubernetes. By the time you reach those tools, you will be packaging and deploying a system whose internals you already understand.

Go Adaptation of the Original Stack

The supplied blueprint chose Python. This guide preserves its architecture and development order, but changes implementation choices to Go at the user's request. These are **implementation adaptations**, not extra product requirements.

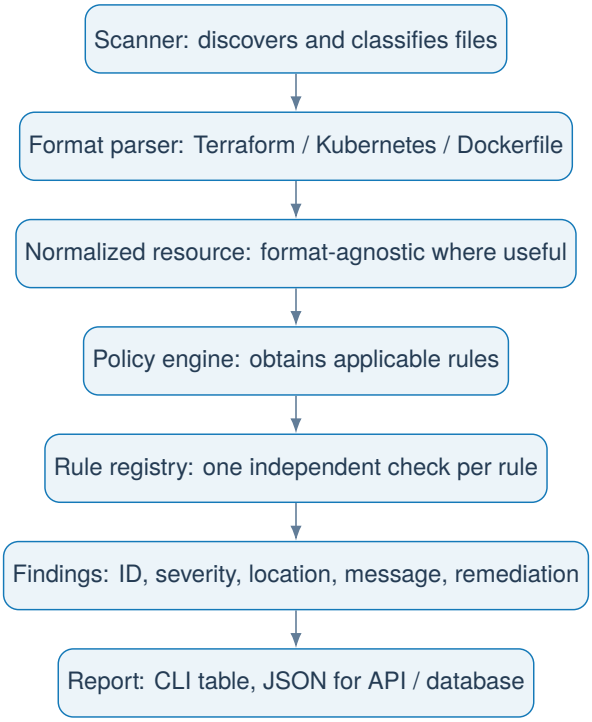
GO-SPECIFIC IMPLEMENTATION ADAPTATIONS
Use Go structs, interfaces, explicit errors, `context.Context`, the standard testing package, and structured logging. Do not search for a one-to-one replacement for every Python library; retain the engineering intent and let the implementation feel native to Go.

Layer	Go choice	Reason / adaptation
Language	Go (current stable release)	Suits infrastructure tooling; use modules, idiomatic package boundaries, explicit errors, and contexts.
HTTP API	`net/http` with a small router such as `chi`	A minimal REST layer; the router is an adaptation, not a core project feature.
Models / validation	Structs, enums, JSON tags; validate request boundaries explicitly	Replaces Pydantic-style models without hiding domain rules in a framework.
Database	PostgreSQL with a Go driver and migrations	Preserve durable audit history and migration discipline; keep queries and repository code simple.
Testing	`go test`, table-driven tests, fixtures, coverage	Replaces pytest while retaining fixture-driven validation.
Quality tools	`gofmt`, `go vet`, `staticcheck` / `golangci-lint`	Format and analyse from the first commit; wire into CI later.
Parsing	HashiCorp HCL, `gopkg.in/yaml.v3`, a small Dockerfile tokenizer	Natural Go choices for Terraform, Kubernetes, and Dockerfile input.
Logging / config	`log/slog`; environment variables	Structured logs and no hard-coded secrets.
Async work	Redis-backed worker, selected only in Phase 5	Preserve the queue pattern; choose a small Go library or a simple worker implementation after synchronous scans work.

Part 2 — Rule Engine Architecture

Pipeline

The scanner core should not know individual rule names. Parsers turn each supported format into a normalized representation; a policy engine requests the applicable rules from a registry; rules produce structured findings; the CLI and API only consume the findings.



Rule Definition Format

Each rule is declarative metadata plus one detection method.

Field	Example: `K8S-001`
Description	Container runs in privileged mode
Severity	`CRITICAL`
Category	Container security
Detection	`securityContext.privileged == true`
Recommendation	Do not use `privileged: true` without a documented, reviewed operational need. Prefer only the specific Linux capabilities required.
Reference	CIS Kubernetes Benchmark 5.2.1 mapping only — not a compliance claim

Listing 1: A deliberately small Go rule interface

```go
type Rule interface {
    ID() string
    Metadata() RuleMetadata
    AppliesTo() FileType
    Evaluate(ctx context.Context, resource NormalizedResource) []Finding
}

type PrivilegedContainerRule struct{}

func (PrivilegedContainerRule) ID() string { return "K8S-001" }
func (PrivilegedContainerRule) AppliesTo() FileType { return Kubernetes }

func (PrivilegedContainerRule) Evaluate(
    ctx context.Context, resource NormalizedResource,
) []Finding {
    var findings []Finding
    for _, container := range resource.Containers {
        if container.SecurityContext.Privileged {
            findings = append(findings, Finding{
                RuleID: "K8S-001", Severity: Critical,
                FilePath: resource.FilePath, Line: container.Line,
                Message: "container " + container.Name + " runs privileged",
                Recommendation: "Remove privileged: true; use explicit capabilities.",
            })
        }
    }
    return findings
}
```

Adding Rules Without Touching the Scanner

The scanner core must never import a specific rule by name.

1. Keep each rule in the Terraform, Kubernetes, or Dockerfile rules package.
2. Register rules explicitly at startup (a Go-friendly registry) rather than using reflection-heavy auto-discovery.
3. Ask the registry for all rules applicable to the current file type, then evaluate them generically.
4. Add a check by adding one rule implementation and its tests, with no edits to the scanner, parser, or CLI.

DESIGN GOAL
This is an open/closed rule system: open for extension, closed for modification. It is the architectural decision that most clearly demonstrates engineering maturity.

Initial Rule Set: Illustrative, Not Exhaustive

Rule ID	Format	Check	Severity
TF-001	Terraform	Security-group ingress from `0.0.0.0/0`	CRITICAL
TF-002	Terraform	SSH (22) open to `0.0.0.0/0`	CRITICAL
TF-003	Terraform	RDP (3389) open to `0.0.0.0/0`	CRITICAL
TF-004	Terraform	Storage without encryption	HIGH
TF-005	Terraform	Publicly readable/writable storage	CRITICAL
TF-006	Terraform	Storage logging/versioning disabled	MEDIUM
TF-007	Terraform	IAM action and resource both wildcarded	HIGH
K8S-001	Kubernetes	`privileged: true`	CRITICAL
K8S-002	Kubernetes	Container may run as root; `runAsNonRoot` missing	HIGH
K8S-003	Kubernetes	CPU or memory limits missing	MEDIUM
K8S-004	Kubernetes	`hostNetwork: true`	HIGH
K8S-005	Kubernetes	`hostPID: true`	HIGH
K8S-006	Kubernetes	`hostPath` volume used	MEDIUM
K8S-007	Kubernetes	Image tag is `:latest` or absent	MEDIUM
K8S-008	Kubernetes	No `securityContext`	LOW
K8S-009	Kubernetes	Broad added capability, for example `ALL` or `SYS_ADMIN`	CRITICAL
DOCKER-001	Dockerfile	No `USER` instruction; container runs as root	HIGH
DOCKER-002	Dockerfile	`FROM ...:latest`	MEDIUM
DOCKER-003	Dockerfile	Secret-looking value in `ENV` or `ARG`	CRITICAL
DOCKER-004	Dockerfile	Remote URL used with `ADD` rather than `COPY`	LOW
DOCKER-005	Dockerfile	Package-manager cache not cleaned	INFO

HONEST SCOPE
Roughly 18–21 rules demonstrate the architecture; they do not provide complete security coverage. Describe DeployShield as a focused portfolio scanner inspired by tools such as Checkov, tfsec, and Trivy — never as their replacement.

Part 3 — Development Phases: 0 to 14

Each phase has a goal, a deliverable, and a boundary. Keep the order. The later phases demonstrate DevSecOps depth, but they should wrap a scanner that already works.

Phase 0 — Project Setup

Item	What to do
Repository	Create `deployshield`, push immediately, add a Go `.gitignore`.
Environment	Initialize `go.mod`; keep dependencies deliberate. Do not over-tool a solo project.
Configuration	Read environment variables; never hard-code DB URLs, tokens, or webhook secrets.
Logging	Configure structured `log/slog` logging once in `internal/platform/logging`.
Errors	Define a small error vocabulary: parse, rule, scan, and storage errors; preserve context with wrapping.
Quality	Use `gofmt`, `go vet`, and a linter from commit one.
README	State the purpose, honest scope, quick-start, architecture, and current status.

Listing 2: Suggested Go-oriented repository shape

```
deployshield/
+-- cmd/deployshield/ # CLI entrypoint
+-- internal/
| +-- core/ # config, logging, domain errors
| +-- parser/ # terraform, kubernetes, dockerfile
| +-- model/ # resource, finding, severity
| +-- rules/ # terraform/, kubernetes/, dockerfile/
| +-- engine/ # registry, policy engine, scanner
| +-- api/ # HTTP handlers (Phase 3)
| +-- storage/ # PostgreSQL + migrations (Phase 4)
| +-- worker/ # async worker (Phase 5)
+-- migrations/
+-- tests/fixtures/
+-- deployments/ # Docker/Kubernetes/Argo artifacts later
+-- go.mod
'-- README.md
```

Phase 1 — Core Static Scanner

Build parser → normalize → evaluate → findings for Terraform, Kubernetes, and Dockerfiles. This is the most important phase.

Format	Parsing approach	Important limitation
Terraform	HashiCorp HCL parser to obtain a syntax/model representation	Variables and references can be dynamic. Analyse literal values; report unresolved dynamic values as `INFO`, never silently skip them.
Kubernetes	`yaml.v3` decoder; a file may contain multiple `---`-separated documents	Preserve positions where practical so findings have useful locations.
Dockerfile	Small line-by-line tokenizer keyed by `FROM`, `RUN`, `USER`, `ENV`, `COPY`, etc.	Correctly handle comments and backslash continuations.

Phase 2 — CLI

The CLI scans recursively, respects exclusions, classifies files, parses, evaluates applicable rules, aggregates results, prints a table, and computes a CI-safe exit code.

Listing 3: Target command-line experience

```
$ deployshield scan ./test-fixtures --fail-on critical
Scanning 7 files...

CRITICAL: 2 HIGH: 3 MEDIUM: 4 LOW: 1 INFO: 0
[CRITICAL] K8S-001 privileged_container.yaml:14 container app runs privileged
[CRITICAL] TF-002 open_security_group.tf:9 SSH open to 0.0.0.0/0

Scan failed: 2 CRITICAL findings exceed policy threshold.
```

Exit	Meaning	Why it matters
0	Scan completed; no configured policy violation	CI succeeds and a PR can proceed.
1	Scan completed; policy violation found	CI fails and can block a merge.
2	Scanner errored (bad input or internal failure)	A crashing scanner must not look like a clean scan.

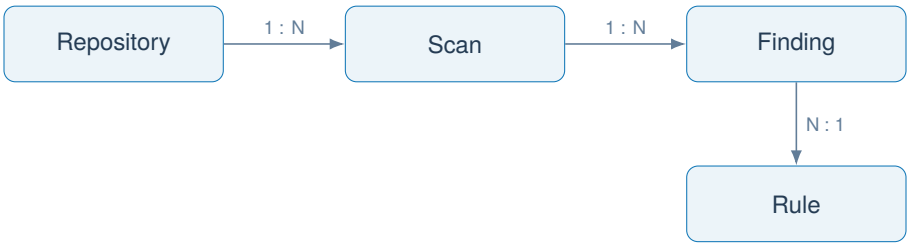
Phase 3 — REST API

Keep scans synchronous in this phase. The API is a thin front door over the same scanner, not a second scanning implementation.

Endpoint	Purpose
`POST /scans`	Submit a scan job using a local path or repository URL plus branch.
`GET /scans`	List scans with status and severity summary.
`GET /scans/{scan_id}`	Return one scan's detail and severity counts.
`GET /scans/{id}/findings`	Return findings, filterable by severity or rule.
`GET /health`	Liveness/readiness endpoint for later container and Kubernetes probes.

Phase 4 — PostgreSQL and Audit History

Persist only what the project needs: repositories, scans, findings, and rule metadata. Do not add users or per-project policies until a later need actually demands them.



Phase 5 — Asynchronous Scanning with Redis

Introduce a queue only after synchronous requests work end-to-end. A worker accepts jobs, clones or reads the target, runs the scanner, writes findings, updates state `QUEUED -> RUNNING -> DONE/FAILED`, and retries only genuinely transient failures such as a network clone error.

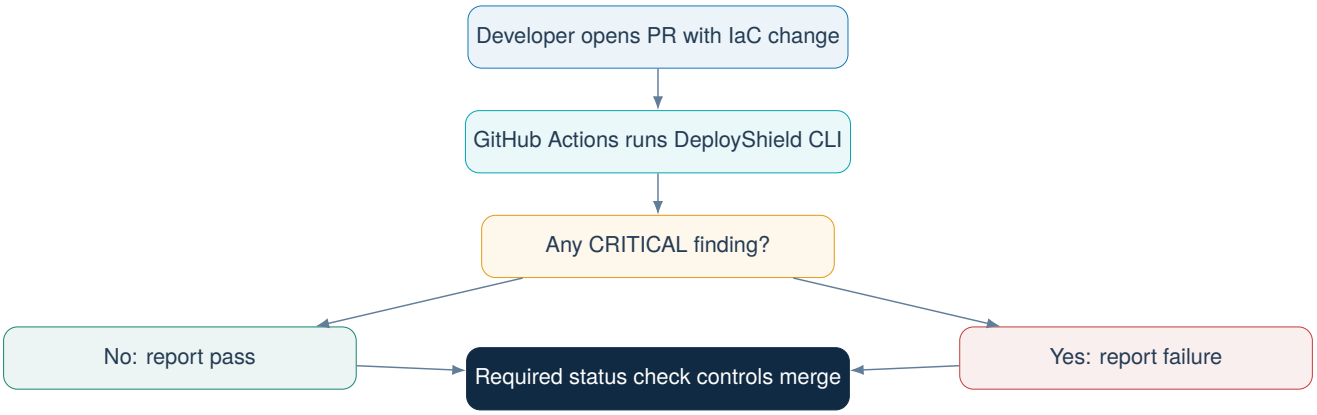
Phase 6 — Docker

Only containerize the API, worker, PostgreSQL, and Redis after they run directly on your machine. Docker provides reproducibility and packaging; it must not conceal an application that only works in a container.

Phase 7 — CI/CD with GitHub Actions

The workflow should: check out code and cache modules; format, vet, and lint; run unit tests; run integration tests with PostgreSQL/Redis services; scan the built image or dependency manifest with Trivy; build the image; and publish a versioned image to GHCR on main-branch merges.

Phase 8 — GitHub Pull-Request Security Gate



The enforcement is the feature: branch protection refuses the merge until a required check passes. A local scan is advisory; a required pull-request check is not.

Phase 9 — OPA / Rego Policy Engine

OPA/Rego is an additional evaluation backend, not a rewrite of the native Go rule engine. Demonstrate policy-as-code by evaluating a small subset of normalized resources through OPA alongside native rules. Keep the first implementation focused on two or three policies.

Phase 10 — Kubernetes

Only after Phases 0–9 work without a cluster should you deploy DeployShield into one. Use kind as the default local cluster because it is disposable and CI-friendly. Learn Pods, Deployments, Services, namespaces, ConfigMaps, Secrets, and `kubectl` basics while deploying the API and worker you already understand.

Phase 11 — Kubernetes Admission Controller

The validating webhook receives an `AdmissionReview` over HTTPS, calls the same policy engine used by the CLI, and returns an allow or deny response with the matching request UID. Implement HTTPS, `ValidatingWebhookConfiguration`, a trusted TLS certificate, explicit `failurePolicy`, minimal privileges, and input validation. `Fail` is more secure but may block deployments during webhook outage; document the choice.

Phase 12 — GitOps with Argo CD

Git becomes the desired-state source of truth. After a PR passes the security gate and merges, Argo CD reconciles the manifest into Kubernetes; the admission webhook performs a final live check. This demonstrates shift-left security plus defense in depth.

Phase 13 — Observability

Expose `/metrics`; scrape it with Prometheus; build two or three Grafana dashboards: Scan Overview, Findings Trend, and System Health.

Metric	Type	Why
Scans total / succeeded / failed	Counter	Basic pipeline health
Scan duration	Histogram	Detect performance regressions as rule count grows
Findings by severity and rule ID	Labeled counter	Security-posture trend and rule tuning
API request latency	Histogram	Standard service-health signal
Queue length	Gauge	Worker backpressure
Worker failures	Counter	Reliability signal

Phase 14 — Terraform for DeployShield's Own Infrastructure

Use Terraform to provision local or inexpensive infrastructure only after the scanner is mature. A good demo is scanning the project's own Terraform in CI, finding a genuine issue, and fixing it.

Part 4 — Your First Session Tonight (2–4 Hours)

SESSION GOAL
`deployshield scan ./tests/fixtures/kubernetes` runs and prints at least one genuine finding. Nothing else is required tonight.

- [ ] Create the GitHub repository, clone it, add a Go `.gitignore`, and make the first commit.
- [ ] Run `go mod init github.com/<your-username>/deployshield`; add only YAML parsing and any small CLI dependency you truly need.
- [ ] Create the Phase 0 folder skeleton.
- [ ] Define `Severity` and `Finding` structs — 15 minutes.
- [ ] Define a minimal Kubernetes-Pod normalized resource with container and security-context fields.
- [ ] Parse one YAML file; pull a Pod's containers and `securityContext` into the normalized resource.
- [ ] Implement `K8S-001` exactly: flag `privileged: true`.
- [ ] Create insecure and clean Pod fixtures.
- [ ] Write a table-driven unit test: insecure fixture gives one `CRITICAL`; clean fixture gives none.
- [ ] Run `go test ./...` until it is green.
- [ ] Add the smallest possible CLI path: discover `.yaml`, run the one rule, print plain-text findings.
- [ ] Commit and push. This is milestone #1.

DO NOT, TONIGHT
Do not install Docker or kind. Do not parse Terraform. Do not build the HTTP API. Do not design the database. One rule, one format, one passing test, one CLI command — that is the entire scope of the first session.

Part 5 — Testing Strategy

Core Idea

Static analysis is tested with files on disk. A hand-written insecure or secure IaC fixture enters the scanner; actual findings are compared with expected findings; the test passes or fails. No live cloud account, running cluster, or container runtime is needed.

Listing 4: Fixture-first test layout in Go

```
tests/
+-- fixtures/
| +-- terraform/ # secure.tf, public_s3.tf, open_security_group.tf
| +-- kubernetes/ # secure.yaml, privileged_container.yaml, insecure_pod.yaml
| `-- dockerfiles/ # secure.Dockerfile, root_user.Dockerfile
+-- parser_test.go
+-- rules_terraform_test.go
+-- rules_kubernetes_test.go
+-- rules_dockerfile_test.go
`-- scan_integration_test.go
```

Every insecure fixture should ideally trip exactly one rule so the test isolates the variable being checked. Include at least one clean fixture per format: a scanner that never reports clean input is as untrustworthy as one that never finds problems.

Layer	What it proves	Example
Parser tests	Raw file becomes the right normalized model	A privileged-Pod fixture has a first container with privileged set.
Rule unit tests	One rule returns the exact expected findings	`K8S-001` returns one `CRITICAL` finding.
Engine/integration tests	Full pipeline works against real files	Scanning the fixture directory returns known severity totals.
API tests	HTTP behaviour and persistence are correct	Submitting a local fixture returns a scan whose stored findings match expectations.
Regression tests	A fixed bug remains fixed	Add a named fixture for every confirmed false positive or parsing bug.

GO TESTING STYLE
Prefer table-driven tests and subtests. The original roadmap's parameterized-test intent maps naturally to a table of `fixturePath`, expected rule ID, expected severity, and expected count.

Part 6 — Recommended Build Order and Four-Month Plan

Dependency-Driven Order

Stage	Deliverable	Depends on
1	Project setup, structure, tooling	—
2	Kubernetes scanner first	1
3	Terraform scanner	2
4	Dockerfile scanner	2
5	Generalized rule engine and registry	2–4
6	Full fixtures, unit/integration/regression suite	5
7	CLI	5–6
8	Synchronous REST API	7
9	PostgreSQL audit history	8
10	Redis and asynchronous workers	9
11	Docker and Docker Compose	10
12	GitHub Actions CI/CD	11
13	Pull-request security gate	12
14	OPA/Rego demonstration	5; demonstrate after 13
15	Kubernetes / kind deployment	11
16	Admission controller	15
17	Argo CD / GitOps	15–16
18	Prometheus / Grafana	15
19	Terraform for DeployShield infrastructure	3, 11

Build Kubernetes scanning first because YAML is the quickest path to a complete vertical slice. This is a sequencing choice; all three formats remain MVP scope.

Four-Month Calendar

Month	Primary outcome	Weeks and focus
1	A tested, usable scanner	W1 setup + one Kubernetes rule; W2 Kubernetes scanner; W3 Terraform and Dockerfile parsing; W4 rule registry, tests, CLI.
2	A persistable scanner service	W5 synchronous API; W6 PostgreSQL/audit history; W7 Redis worker; W8 Docker Compose and end-to-end hardening.
3	Delivery security and policy-as-code	W9 GitHub Actions; W10 required PR gate; W11 OPA/Rego subset; W12 documentation, regression tests, and demo quality.
4	Advanced DevSecOps demonstration	W13 kind deployment; W14 admission webhook; W15 Argo CD plus metrics/dashboards; W16 Terraform self-infra, final demo, and CV evidence.

SCOPE CONTROL
If Month 2 slips, protect the MVP: scanner, registry, tests, CLI, API, database, and CI. Advanced infrastructure is valuable only after the core remains credible and demonstrable.

Part 7 — Technology Learning Map

Technology	Learn now	Explicitly defer	Project task
Go	structs, interfaces, errors, contexts, modules, tests	advanced reflection, premature concurrency patterns	parsers and rules
HTTP API	handlers, JSON, middleware basics, request validation	WebSockets, GraphQL	Phase 3 endpoints
PostgreSQL	SQL, foreign keys, migrations, basic connection use	partitioning, replication, deep tuning	Phase 4 schema
Redis	queue/worker pattern and delivery implications	unrelated pub-sub or cache features	Phase 5 worker
Docker	images, containers, Compose networks/volumes, env injection	micro-optimised builds and custom runtimes	Phase 6 Compose file
GitHub Actions	workflow YAML, jobs, services, secrets	self-hosted runners and large reusable-workflow libraries	Phase 7 pipeline
Terraform	HCL, resources/providers, state basics	enterprise features and complex module registries	fixtures, then Phase 14
Kubernetes	Pods, Deployments, Services, namespaces, ConfigMaps, Secrets	multi-cluster, service mesh, unrelated CRDs	Phase 10 deployment
OPA/Rego	basic `deny[msg]` rules and one integration path	full standard library and partial evaluation	Phase 9 subset
Argo CD	application, sync policy, Git as source of truth	ApplicationSets and multi-cluster topology	Phase 12
Prometheus + Grafana	counters, histograms, gauges, scrape config, basic PromQL	long-term storage, federation, plugins	Phase 13 dashboards

Part 8 — Project Management Practices

Practice	Recommendation
Branching	Trunk-based: short-lived feature branches from `main`, merged through pull requests. GitFlow is unnecessary for this solo project.
Commit messages	Conventional Commits: `feat:`, `fix:`, `test:`, `docs:`, `refactor:`.
Issue tracking	One GitHub Issue per rule or feature; label by phase and type (bug, rule, infra).
Milestones	One milestone per Phase 0–14; close only when its checklist is genuinely complete.
Pull requests	Open a PR for every non-trivial change, even solo, so Phase 8 can be demonstrated on your own repository.
Review	Self-review the PR diff. Ask a peer to review a few high-value PRs, such as the registry or webhook.
Versioning	Semantic Versioning: `v0.1.0` at first working scanner; `v1.0.0` at MVP completion.
Changelog	Maintain `CHANGELOG.md` in Keep a Changelog style.
Documentation	Keep `README.md`, `docs/architecture.md`, and `docs/adr/`.
ADRs	Record debatable choices: Go implementation adaptation, native rules vs OPA-only, kind vs minikube, synchronous before asynchronous scans.

Part 9 — Security of DeployShield Itself

CENTRAL RISK
DeployShield scans other repositories. Treat every scanned repository and every submitted target as hostile input.

Concern	Mitigation
Repository URL validation	Allow-list expected hosts such as `github.com`; reject `file://` and private/internal network targets to prevent SSRF.
Command injection	Do not compose shell strings from paths or URLs. In Go, use `exec.CommandContext` with fixed arguments and no shell.
Unsafe parsing	Parse YAML as data only; never execute custom tags or scan-time code. Parse HCL; never evaluate Terraform.
Executing scanned code	Never run `terraform apply`, build images, or execute scripts from a scanned repository. This stays static analysis.
Secrets	Inject DB credentials and tokens through environment variables or a secret store; do not log full request bodies.
Authentication	Require API keys or JWT before exposure beyond localhost.
Authorization	When multi-user support exists, scope scan history to the user or organisation that submitted it.
Worker isolation	Clone and parse untrusted repositories in a resource-limited, network-restricted, minimally privileged worker.
Container security	Run DeployShield containers non-root with minimal images and no unneeded capabilities. Practise what `DOCKER-001` checks.
Webhook security	HTTPS is mandatory; validate inputs, minimise privileges, and make failure behaviour explicit.
Least privilege	The worker and admission-webhook service accounts receive only the permissions each actually requires.

Part 10 — Real-World Positioning and Compliance

This Tool Category Is Real

Tool	Category	General role
Checkov	IaC static analysis	Terraform, CloudFormation, Kubernetes, and Dockerfile misconfiguration scanning.
tfsec / Trivy	IaC / broad scanning	Terraform scanning and broader vulnerability/misconfiguration scanning.
KICS	Multi-IaC scanner	IaC misconfiguration scanning across several formats.
Snyk	Commercial platform	Vulnerability, IaC, and container security in delivery workflows.
OPA / Gatekeeper	Policy engine / admission	Policy-as-code and Kubernetes admission enforcement.
Kyverno	Kubernetes admission	Kubernetes-native policy system using YAML policies.

POSITION IT CORRECTLY
DeployShield is a portfolio implementation demonstrating IaC security, policy-as-code, CI security gates, admission control, and compliance-automation concepts. It is not equivalent to any mature tool listed above.

Who Needs This

- A DevOps or platform engineer needs IaC changes reviewed automatically before merge.
- A cloud engineer opens SSH to the internet in Terraform; the PR gate catches it before apply.
- A DevSecOps engineer moves checks into the pipeline rather than relying on periodic manual reviews.
- An SRE attempts to apply a privileged pod; the admission webhook rejects it before it runs.

Compliance Disclaimer: Use Precise Language

Framework	Relation to DeployShield
CIS Benchmarks	Specific configuration recommendations that a rule-based scanner can map checks to.
PCI DSS	A payment-card standard involving technical, process, organisational, and audit requirements. A scanner alone cannot make an organisation compliant.
SOC 2	An auditor's attestation about controls over time, not a status awarded by a tool.
NIST 800-53	Broad control catalogue; selected technical controls may be partially checkable, but NIST compliance is organisational.

THE DISTINCTION TO REMEMBER
Correct: "DeployShield includes checks mapped to selected CIS Kubernetes Benchmark controls."
Incorrect: "DeployShield is PCI-DSS compliant" or "DeployShield makes infrastructure SOC 2 compliant."

Part 11 — MVP vs Advanced Scope

MVP — Must Finish	Advanced — Stretch in this priority order
☐ Terraform scanning	☐ Redis and async workers
☐ Kubernetes scanning	☐ Docker and Docker Compose
☐ Dockerfile scanning	☐ GitHub PR security gate
☐ Registry-based rule engine	☐ OPA/Rego integration
☐ Severity system and exit codes	☐ kind deployment
☐ CLI	☐ Admission controller
☐ Full test suite and fixtures	☐ Argo CD / GitOps
☐ REST API	☐ Prometheus and Grafana
☐ PostgreSQL persistence	☐ Terraform for DeployShield infrastructure
☐ GitHub Actions CI	

The MVP is already an interview-worthy backend and engineering project. If time runs out, MVP plus Docker and CI is a coherent, defensible portfolio outcome. Do not describe unfinished advanced features as delivered work.

Part 12 — Common Mistakes to Avoid

- ☐ Building every layer at once instead of following the dependency order.
- ☐ Learning all of Kubernetes, Docker, and Terraform before writing the first scanner rule.
- ☐ Writing a huge ruleset before the registry and base rule contract are solid.
- ☐ Claiming compliance without organisational authority or evidence.
- ☐ Copying another scanner's logic without understanding the security reason behind a check.
- ☐ Adding users, plugin systems, or a polished UI before the scanner works.
- ☐ Skipping tests to move faster. Untested security rules damage credibility.
- ☐ Deploying to paid cloud infrastructure before it creates real value.
- ☐ Listing features on a CV that do not run end-to-end in a live demonstration.

Part 13 — CV Value

Skills Demonstrated

Software engineering	DevOps / cloud / security
Modular open/closed rule-engine design	GitHub Actions CI/CD and required security checks
Go CLI and REST API design	Docker and Docker Compose
PostgreSQL schema design and migrations	Kubernetes fundamentals and admission control
Async job processing and structured errors/logs	Terraform and GitOps with Argo CD
Fixture-driven testing and regression protection	Policy-as-code with OPA/Rego
Native policy engine integration	Prometheus/Grafana operational observability

CV-Ready Description

DeployShield — Infrastructure-as-Code Security Scanner & DevSecOps Policy Engine
Go, PostgreSQL, Docker, Kubernetes, Terraform, GitHub Actions

- Designed and built a Go static-analysis engine that scans Terraform, Kubernetes, and Dockerfile configurations for security misconfigurations using an extensible rule registry with focused checks across `CRITICAL`–`INFO` severities.
- Implemented a CLI and REST API for on-demand scans, with PostgreSQL audit history for scans and findings.
- *Only if completed:* Added Redis-backed asynchronous scanning to separate API responsiveness from scan duration.
- *Only if completed:* Containerized the system with Docker Compose and implemented GitHub Actions CI including formatting, tests, image scanning, and automated publishing.
- *Only if completed:* Added a required pull-request security gate that blocks merges on `CRITICAL` findings.
- *Only if completed:* Built a Kubernetes validating admission webhook that enforces the same policies during deployment, with a small OPA/Rego policy subset.
- *Only if completed:* Deployed through Argo CD with Prometheus/Grafana dashboards for throughput, latency, and findings trends.

NON-NEGOTIABLE CV RULE
Keep only bullets matching features that genuinely run end-to-end and that you can demo live. Delete every bracketed bullet for a phase you did not reach.

Part 14 — Success Criteria and Master Checklist

MVP Success

- [ ] Scanner finds at least 10–15 distinct security issues across the three formats.
- [ ] Known false positives are documented in the README rather than hidden.
- [ ] Tests cover every implemented rule and the parser layer.
- [ ] CLI produces correct CI-usable exit codes.
- [ ] API persists and retrieves scans and findings correctly.
- [ ] CI passes on good code and fails on bad code; test both directions.

Advanced Success

- [ ] A genuine PR receives automated pass/fail security feedback.
- [ ] A `CRITICAL` change is blocked by branch protection.
- [ ] The admission webhook rejects a disallowed Pod applied with `kubectl`.
- [ ] Grafana displays real scan metrics, not placeholder data.
- [ ] API, worker, database, and queue start reproducibly with one command on a clean machine.

Final Master Checklist

- [ ] Repository and Go module created
- [ ] Configuration, logging, errors
- [ ] Scanner architecture implemented
- [ ] Terraform parser
- [ ] Kubernetes parser
- [ ] Dockerfile parser
- [ ] Rule registry and severity system
- [ ] Unit and integration tests
- [ ] CLI
- [ ] HTTP API
- [ ] PostgreSQL and migrations
- [ ] Live demo and honest CV update
- [ ] Redis and async workers
- [ ] Docker and Docker Compose
- [ ] GitHub Actions CI/CD
- [ ] PR gate / branch protection
- [ ] OPA/Rego subset
- [ ] kind cluster
- [ ] Admission-controller webhook
- [ ] Argo CD / GitOps
- [ ] Prometheus and Grafana
- [ ] Terraform for DeployShield infrastructure
- [ ] README, architecture docs, ADRs

Start with Part 4 tonight. Return to Part 2 every time you add a new rule.